

SE LAB-04

NAME: Diya R Gowda

SECTION: 5 'C'

SRN: PES1UG24CS159

The chat link: <https://claude.ai/share/eb01df76-ffcd-468e-911d-850b05ae5f86>

Prompt 1

I'm working on a single-file Python 3.14 Pygame project (Galaga-lite clone, game.py). It's a lab where I have to find and fix a deliberate bug.

Problem: The game runs without errors, but enemy entry and dive paths, which are cubic Bézier curves, bulge or overshoot mid-flight and look wrong. The start and end points are still correct. I suspect the bezier() function has a wrong term or exponent.

Expected: The curves should follow the standard cubic Bézier formula: $(1-t)^3 \cdot P_0 + 3(1-t)^2 \cdot t \cdot P_1 + 3(1-t) \cdot t^2 \cdot P_2 + t^3 \cdot P_3$, passing smoothly through the two control points.

Please:

1. Compare each of the four terms in bezier() against the standard formula and explain the bug briefly.
2. Give me only the corrected bezier() function.
3. Don't change anything unrelated.

Here is the full code:

Answer 1

Term-by-term comparison

Term	Standard	In your code	Status
1	$(1-t)^3 \cdot P_0$	<code>u ** 3 * p0</code>	Correct
2	$3(1-t)^2 \cdot t \cdot P_1$	<code>3 * u * u * t * p1</code>	Correct
3	$3(1-t) \cdot t^2 \cdot P_2$	<code>3 * u * t * p2</code>	Bug: t should be squared
4	$t^3 \cdot P_3$	<code>t ** 3 * p3</code>	Correct

The third term is missing its square on t, in both the x and y lines. That breaks the Bernstein weights, which must sum to 1 for every t. With the bug, the weights at $t = 0.5$ sum to 1.375, so the curve is pulled outward and overshoots mid-flight. The endpoints still look right

because the third term vanishes at both $t = 0$ (from t) and $t = 1$ (from u), which is why the start and end points weren't affected.

Corrected function

python

```
def bezier(p0, p1, p2, p3, t):
```

```
    u = 1 - t
```

```
    x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * t * p2[0] + t ** 3 * p3[0]
```

```
    y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * t * p2[1] + t ** 3 * p3[1]
```

```
    return pygame.Vector2(x, y)
```

Nothing else was changed.

Prompt 2

The bezier fix works. Here is my current code:

Now implement the stub `enemy_tint(kind)`. It's called once per visible enemy per frame in `draw()` as:

```
color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]
```

`kind` is "boss", "red", or "blue". It should return an (r, g, b) tuple, or None to keep the default color.

Feature: recolor enemies by wave number so later waves look visually distinct.

Requirements:

- The function only receives `kind`, so use a module-level variable such as `CURRENT_WAVE` that is updated when a new wave starts (`spawn_wave` already receives the wave number). Tell me exactly where you set it.
- Waves 1 and 2 keep the default colors (return None). From wave 3 onward, use a different palette that changes every 2 waves and cycles, keeping the three kinds distinguishable.
- Keep the existing damaged-boss purple color logic in `draw()` working.
- Keep all existing functionality working.
- Show me only the changed or new lines/functions and say where they go.

Answer 2

Two pieces change: a new module-level variable plus the `enemy_tint` implementation, and one line added to `spawn_wave`.

1. Replace the `enemy_tint` stub (keep it in the same spot, and put `CURRENT_WAVE` and `WAVE_PALETTES` directly above it):

```
python
```

```
CURRENT_WAVE = 1
```

```
# Each palette maps every kind to a distinct colour. Palette changes every 2 waves from wave 3.
```

```
WAVE_PALETTES = [  
    {"boss": (255, 200, 60), "red": (240, 120, 40), "blue": (60, 200, 200)}, # waves 3-4  
    {"boss": (255, 90, 200), "red": (170, 60, 240), "blue": (90, 230, 130)}, # waves 5-6  
    {"boss": (240, 240, 100), "red": (60, 220, 170), "blue": (250, 110, 110)}, # waves 7-8  
]
```

```
def enemy_tint(kind):
```

```
    """Return an (r, g, b) colour override for an enemy kind, or None for the default."""
```

```
    if CURRENT_WAVE < 3:
```

```
        return None
```

```
    palette = WAVE_PALETTES[((CURRENT_WAVE - 3) // 2) % len(WAVE_PALETTES)]
```

```
    return palette.get(kind)
```

2. In `spawn_wave`, add the global declaration and the assignment. The updated function:

```
python
```

```
def spawn_wave(wave):
```

```
    global CURRENT_WAVE
```

```
    CURRENT_WAVE = wave
```

```
    enemies, index = [], 0
```

```
    layout = [("boss", 4, 90), ("red", 8, 130), ("red", 8, 170), ("blue", 8, 210)]
```

```

for kind, count, y in layout:
    for i in range(count):
        x = WIDTH / 2 + (i - (count - 1) / 2) * 52
        enemies.append(Enemy(kind, (x, y), index, index % 2 == 0))
        index += 1
    on_wave_start(wave)
return enemies

```

The only new lines are global `CURRENT_WAVE` and `CURRENT_WAVE = wave` at the top of the function body. Everything else in it is unchanged.

Why this location works

- `spawn_wave` is called from both `Game.reset()` (with wave 1) and the wave-advance branch in `Game.update()`, so `CURRENT_WAVE` is set correctly on a new game, after pressing R, and on every subsequent wave. No changes to `Game` are needed.
- Waves 1-2 return `None`, so `draw()` falls back to `ENEMY_COLORS`.
- The purple damaged-boss override in `draw()` runs after the `enemy_tint(...)` or ... line, so it still takes precedence.
- Wave 9 wraps back to the first palette, and so on. Each palette uses three clearly different hues so boss, red, and blue stay distinguishable, and none of them uses the damaged-boss purple (170, 90, 220). The palette 2 red (170, 60, 240) is somewhat close to it though, so if you find it confusing against a damaged boss, swap it for another colour.

Prompt 3

`enemy_tint` works. Here is my current code:

Now implement the stub `on_wave_start(wave)`. It's called once from `spawn_wave(wave)`, right after the enemy list is built, at game start and every time a wave is cleared. Its return value is ignored.

Feature: show a "WAVE N" banner for about 1.5 seconds at the start of each wave.

Requirements:

- The function has no access to the `Game` object, so use a module-level variable for the banner timer and wave number, set inside `on_wave_start`.

- Decrement the timer in `Game.update()` using `dt`, and draw the banner centered on the screen in `Game.draw()` with a larger font (create it in `draw` or `main`).
- The banner must also appear on wave 1 and after pressing R to reset.
- Do not pause gameplay while the banner shows.
- Keep all existing functionality working.
- Show me only the changed or new lines/functions and say exactly where each goes.

Answer 3

Four small edits: a module-level banner state, the `on_wave_start` body, one line in `Game.__init__`, and the timer and drawing code in `Game`.

1. Replace the `on_wave_start` stub with this (same spot, banner variables directly above it):

python

```
BANNER_TIME = 1.5
```

```
banner_timer = 0.0
```

```
banner_wave = 1
```

```
def on_wave_start(wave):
```

```
    """Called at the start of every wave; add banners, speed-ups, or palette swaps here."""
```

```
    global banner_timer, banner_wave
```

```
    banner_wave = wave
```

```
    banner_timer = BANNER_TIME
```

2. In `Game.__init__`, add one line before `self.reset()`:

python

```
    self.banner_font = pygame.font.Font(None, 72)
```

`Game()` is created in `main()` after `pygame.init()`, so the font system is ready. Building the font here also avoids recreating it every frame and keeps the `draw()` signature unchanged.

3. In `Game.update`, add a global line as the first line of the method body, and the decrement right after `self.time += dt`:

python

```
def update(self, dt, keys):  
    global banner_timer  
    if self.state != "play":  
        return  
    self.time += dt  
    banner_timer = max(0.0, banner_timer - dt)  
    self.cooldown -= dt  
    ...
```

Only the global banner_timer line and the banner_timer = ... line are new. The rest of the method is unchanged.

4. In Game.draw, add this at the very end of the method, after the if self.state == "lose": block:

python

```
if banner_timer > 0 and self.state == "play":  
    text = self.banner_font.render(f"WAVE {banner_wave}", True, (255, 255, 120))  
    screen.blit(text, text.get_rect(center=(WIDTH // 2, HEIGHT // 2)))
```

Why this covers your requirements

- **Wave 1 and reset:** Game.reset() calls spawn_wave, which calls on_wave_start. That happens in Game.__init__ at launch and again whenever you press R, so the banner shows in both cases.
- **Later waves:** the wave-advance branch at the end of Game.update calls spawn_wave, which sets the timer to 1.5 seconds. The next frame's decrement counts it down.
- **No pause:** the banner is only drawn and counted down. Nothing in the gameplay logic checks it, so enemies, bullets and input keep running.
- **Game over:** the self.state == "play" check in draw() keeps the banner from overlapping the "GAME OVER" text if you die while it is showing.

Prompt 4

The wave banner works. Here is my current code:

Now implement the stub `shield_charges(wave)`. It's called as `self.shield = shield_charges(self.wave)` or 0 at game start and at the start of every new wave. It should return the number of hits the player can absorb this wave, or `None/0` to disable the shield. `hit_player()` already consumes charges.

Feature: grant 1 shield charge every 3 waves (waves 3, 6, 9, ...).

Requirements:

- Return `None` on other waves.
- Also show the remaining shield charges in the HUD text in `draw()`, for example "Shield 1", and draw a thin cyan circle around the player ship(s) while `self.shield > 0`.
- Keep all existing functionality working.
- Show me only the changed or new lines/functions and say exactly where each goes.

Answer 4

Three edits.

1. Replace the `shield_charges` stub (same spot in the file):

python

```
def shield_charges(wave):
```

```
    """Return how many hits the player's shield can absorb this wave, or None to disable the shield."""
```

```
    if wave % 3 == 0:
```

```
        return 1
```

```
    return None
```

2. In `Game.draw`, replace the `hud = font.render(...)` line with:

python

```
    hud = font.render(f"Score {self.score} Lives {self.lives} Wave {self.wave} Shield {self.shield} R = reset", True, (240, 240, 240))
```

3. In `Game.draw`, add this right after the ship-drawing if `self.invulnerable <= 0` or ...: block and before the `hud = ...` line. Keep it at the same indentation as that if, not inside it:

python

```
    if self.shield > 0:
```

```
for sx in self.ship_xs():  
    pygame.draw.circle(screen, (0, 255, 255), (int(sx), PLAYER_Y - 3), 24, 1)
```

Notes

- The circle is drawn outside the blink condition on purpose, so it stays visible while the ship flickers. When a hit consumes the charge, `hit_player()` drops `self.shield` to 0 and the circle disappears on the next frame.
- The circle is drawn around each ship in `ship_xs()`, so it covers both ships after a boss capture-style power-up.
- Both `Game.reset()` and the wave-advance branch in `Game.update()` already do `self.shield = shield_charges(self.wave) or 0`, so no changes are needed there. That assignment overwrites the value, so an unused charge from wave 3 does not carry into wave 4. If you'd rather have charges accumulate, change those two lines to add instead of assign (in reset it should stay an assignment).

Prompt 5

this is my latest code below please check if it is correct completely

Answer 5

I read through the whole file and found no bugs. Everything you've added hangs together, and nothing existing was broken. I haven't run it, so this is a code read, not a playtest.

What I checked

- **bezier():** All four terms match the standard cubic formula, in both x and y.
- **enemy_tint() and CURRENT_WAVE:** `spawn_wave` declares global and sets the value before enemies are built. Waves 1-2 return None, and the palette index `((wave - 3) // 2) % 3` cycles correctly. The damaged-boss purple in `draw()` still overrides the tint because it runs after that line.
- **Banner:** `on_wave_start` sets the timer and wave number. `update()` declares global `banner_timer` before using it. The banner is drawn only while `state == "play"`, and it shows at launch, after R, and on every new wave. Gameplay is never paused.
- **Shield:** `shield_charges()` returns 1 on waves 3, 6, 9... and None otherwise, and or 0 in both call sites handles the None. The HUD text, the cyan circle drawn per ship, and `hit_player()` consuming the charge all line up. The circle disappears on the frame after the charge is used.

- **Multi-ship case:** After a boss kill gives you 2 ships, the shield circle, bullets, collision checks, and screen-edge clamp all still use `ship_xs()` and `span` consistently.

Cosmetic points, none of them bugs

- **Palette clash:** The palette 2 red (170, 60, 240) is close to the damaged-boss purple (170, 90, 220), as mentioned earlier. It's harmless but could be confusing on waves 5-6.
- **Shield carryover:** An unused shield charge does not carry into the next wave, because the value is assigned, not added.
- **Unused parameter:** `player_x` in `Enemy.update()` is never used. It was in the original code, so I left it alone.
- **Trailing whitespace:** Two lines have it, `CURRENT_WAVE = wave` and the `hud = ...` line. It's harmless, but your editor may flag it.

If this is your lab submission, the original deliberate bug (the missing `t` square in `bezier()`) is fixed, so you should be good.